

# Seminar Report

## Team Members: Group 32

Abdur Rehman Khalid

Arnab Kumar Ghosh

Dip Chowdhury

Muhammad Umair

Prodip Kumar Das

Zihao Yang

March 2, 2026

## Contents

|          |                                                                      |          |
|----------|----------------------------------------------------------------------|----------|
| <b>1</b> | <b>Question 1: Batch vs. Streaming Processing</b>                    | <b>2</b> |
| <b>2</b> | <b>Question 1: Batch vs. Streaming Processing</b>                    | <b>3</b> |
| <b>3</b> | <b>Question 2: Data Formats and Storage</b>                          | <b>3</b> |
| <b>4</b> | <b>Question 3: The Big Data Ecosystem – Is There a Logic?</b>        | <b>4</b> |
| 4.1      | The Volume Problem: Disk-Bound Batch Processing . . . . .            | 4        |
| 4.2      | The Random Access Problem: Wide-Column Stores . . . . .              | 4        |
| 4.3      | The Velocity Problem: The Streaming Paradigm . . . . .               | 5        |
| 4.4      | The Iteration Problem: In-Memory Unification . . . . .               | 5        |
| 4.5      | Modern Convergence: Cloud-Native and Lakehouse Architectures . . . . | 5        |
| <b>5</b> | <b>Question 4: The Rise of NoSQL</b>                                 | <b>5</b> |
| <b>6</b> | <b>Question 5: Scalability – Theory and Practice</b>                 | <b>6</b> |

# 1 Question 1: Batch vs. Streaming Processing

From a problem perspective, the choice between batch and stream processing is dictated by the nature of the data and latency requirements of the application. Batch processing assumes the existence of static, bounded datasets where all input data are available before the computation begins [1]. This is necessary for scenarios where system throughput and computation is preferred over system latency, such as training language models, analyzing large scale documents where maximizing overall system output and robust fault tolerance are more critical than response time. On the other hand, streaming processing is required for continuous and unbounded data such as server log monitoring, internet-of-things (IoT) sensor outputs or social media where analytics need to be performed continuously in real-time. For example, Twitter uses real-time streaming to measure real-time active user count (RTAC) indicator to find user engagement events [2] where low latency is a strict necessity.

Early batch frameworks like Google’s MapReduce [3] achieved massive scalability and fault-tolerance by using high-throughput disk-based storage. While this ensures reliability against machine failures, it incurs significant disk I/O latency overhead, making the framework unsuitable for real-time queries. To tackle the latency issue, Apache Storm was introduced, which process data one at a time using a directed graph [4]. To address the limitations of Storm, Twitter developed Heron which adopted a one task per JVM model to achieve process isolation and a backpressure mechanism. However, early streaming systems often traded off the consistency and high throughput characteristic of batch processing systems making the latter still relevant.

Some recent frameworks have been developed to connect the gap between batch and streaming processing by unifying the architectural division. Apache Spark approaches this unification through ”micro-batching”, dividing the continuous streams into small batches to leverage fault-tolerance and in-memory batch execution [1]. While Apache Flink took the inverse approach by treating batch processing as streaming processing where data stream is considered finite [5]. Further blurring the lines between the two paradigms, abstraction model like Apache Beam [6] allows programmer to write unified and portable data processing pipelines that can be executed on either batch or streaming processing engines.

Finally, the Dataflow Model [7] serves as the definitive framework for unifying batch and streaming processing to achieve batch processing correctness within a streaming processing environment. Ultimately, the model suggests a trade-off between correctness, latency, and cost, allowing a single pipeline to produce both low-latency high-accuracy results, effectively treating batch processing as a specific instance of streaming where the data source happens to be finite.

When looking at the trade-offs between batch and streaming, the deciding factor is usually the ”**half-life**” of the data’s usefulness. Traditional batch systems, epitomized by **MapReduce**, are engineered for processing and generating large-scale datasets through a functional model that automatically handles parallelization, data distribution, and fault tolerance at scale[cite: 1, 12, 16]. This abstraction is ideal for high-throughput processing on bounded, static datasets; for instance, predicting inventory for a fresh food delivery service based on historical sales is highly efficient via an overnight MapReduce job, which can scale to process terabytes of data across thousands of commodity machines. However, low-latency processing becomes necessary when a system must react to **unbounded data** instantly to prevent operational failure. Frameworks like **Spark** improved upon MapRe-

duce by utilizing in-memory *Resilient Distributed Datasets (RDDs)*, enabling faster iterative processing and **micro-batching**. Yet, for truly dynamic operations—like rerouting delivery drivers during a sudden traffic jam—even a micro-batch delay of a few minutes renders the insight obsolete. The absolute necessity for **strict low-latency streaming** emerges in safety-critical contexts, such as **MRI-guided radiotherapy** (e.g., the TrackRAD challenge), where the system must track a moving tumor in real-time to adjust the radiation beam. In such scenarios, a fraction of a second of latency means irradiating healthy tissue instead of the target. Ultimately, moving beyond batch processing to a continuous streaming architecture introduces massive engineering complexity, especially regarding **state management**; therefore, it should be reserved for use cases where a delayed response is fundamentally an *incorrect* response, while batch processing remains the more stable and cost-effective choice for all other applications.

## 2 Question 1: Batch vs. Streaming Processing

When looking at the trade-offs between batch and streaming, the deciding factor is usually the “**half-life**” of the data’s usefulness. Traditional batch systems, epitomized by **MapReduce**, are engineered for processing and generating large-scale datasets through a functional model that automatically handles parallelization, data distribution, and fault tolerance at scale[cite: 1, 12, 16]. This abstraction is ideal for high-throughput processing on bounded, static datasets; for instance, predicting inventory for a fresh food delivery service based on historical sales is highly efficient via an overnight MapReduce job, which can scale to process terabytes of data across thousands of commodity machines. However, low-latency processing becomes necessary when a system must react to **unbounded data** instantly to prevent operational failure. Frameworks like **Spark** improved upon MapReduce by utilizing in-memory *Resilient Distributed Datasets (RDDs)*, enabling faster iterative processing and **micro-batching**. Yet, for truly dynamic operations—like rerouting delivery drivers during a sudden traffic jam—even a micro-batch delay of a few minutes renders the insight obsolete. The absolute necessity for **strict low-latency streaming** emerges in safety-critical contexts, such as **MRI-guided radiotherapy** (e.g., the TrackRAD challenge), where the system must track a moving tumor in real-time to adjust the radiation beam. In such scenarios, a fraction of a second of latency means irradiating healthy tissue instead of the target. Ultimately, moving beyond batch processing to a continuous streaming architecture introduces massive engineering complexity, especially regarding **state management**; therefore, it should be reserved for use cases where a delayed response is fundamentally an *incorrect* response, while batch processing remains the more stable and cost-effective choice for all other applications.

## 3 Question 2: Data Formats and Storage

In distributed data processing, disk I/O and network bandwidth are primary bottlenecks. Consequently, the choice of data format dictates not only storage efficiency but also CPU overhead and query latency, fundamentally shaping the technological architecture of big data solutions.

**Row vs. Columnar and Text vs. Binary:** Row-oriented, text-based formats like CSV store data sequentially by record. While highly usable due to their human-readability and simplicity for initial data ingestion, they are severely penalized in analyt-

ical (OLAP) workloads. Text parsing is CPU-intensive, and querying a single attribute requires reading entire records from the disk.

Conversely, columnar binary formats like **Apache Parquet** are heavily optimized for analytics [8]. By storing values of the same column contiguously, Parquet enables engines like Spark to perform *projection pushdown*—reading only the specific columns required for a query and entirely skipping irrelevant data. Furthermore, binary formats avoid the overhead of string parsing and allow for aggressive, type-specific compression techniques (such as Run-Length Encoding and dictionary encoding), drastically reducing the disk footprint and I/O wait times [8].

**Schema-on-Read vs. Schema-on-Write:** Traditional relational databases mandate “schema-on-write,” requiring data to be validated and structured during ingestion. As noted in literature discussing the “data deluge,” this loading phase acts as a severe bottleneck, heavily penalizing the ingestion speed of massive scientific datasets [9]. To bypass this, early big data architectures shifted to “schema-on-read,” storing raw files in data lakes and applying structure only at query time. While this maximized ingestion speed and flexibility for exploratory analysis, it often sacrificed query performance and data integrity.

**The Landscape Shift and the Data Lakehouse:** Formats like Parquet and Avro changed this landscape by embedding self-describing schema metadata directly within the binary files. Avro provides highly efficient row-based serialization ideal for streaming, while Parquet dominates analytical reads.

The concept of the **data lakehouse** (e.g., Delta Lake) represents the next evolutionary step. It builds upon open columnar formats like Parquet by adding a transactional metadata layer. This brings “schema-on-write” benefits—such as ACID transactions, strict schema enforcement, and data mutation capabilities—directly to the raw storage layer. Ultimately, the lakehouse bridges the divide, offering the high performance and reliability of a traditional data warehouse alongside the infinite scalability of a data lake.

## 4 Question 3: The Big Data Ecosystem – Is There a Logic?

The big data ecosystem is not fragmented. It is a coherent, reactive progression. Each generation of technologies emerged to solve the critical bottlenecks—volume, latency, or system coupling—of its predecessors.

### 4.1 The Volume Problem: Disk-Bound Batch Processing

Traditional relational databases could not scale horizontally. Google introduced GFS and MapReduce [3], later open-sourced as Apache Hadoop and HDFS, to handle massive data volumes on commodity hardware. But Hadoop had a fundamental flaw: aggressive disk I/O. MapReduce wrote intermediate results to disk after every computation phase. This made it highly fault-tolerant but terrible for low-latency queries and iterative machine learning [10].

### 4.2 The Random Access Problem: Wide-Column Stores

HDFS was optimized for sequential scans of large files. It could not serve real-time point queries. Google BigTable [11] solved this as a distributed NoSQL layer over GFS. This inspired Apache HBase for Hadoop. Facebook then built Apache Cassandra to

eliminate HBase’s centralized master node. Cassandra combined BigTable’s column-family model with Dynamo’s masterless architecture. It traded strong consistency for extreme availability and zero single points of failure.

### **4.3 The Velocity Problem: The Streaming Paradigm**

As data velocity increased, batch processing became too slow. Twitter Storm pioneered distributed streaming but had severe centralized bottlenecks. Twitter Heron redesigned Storm to fix this by decoupling the scheduling framework. Meanwhile, LinkedIn introduced Apache Kafka [12] to prevent high-velocity data from overwhelming processing engines. Kafka acted as a distributed log broker, decoupling data ingestion from computation.

### **4.4 The Iteration Problem: In-Memory Unification**

Apache Spark [10] directly addressed Hadoop’s disk bottleneck with Resilient Distributed Datasets (RDDs). This enabled fault-tolerant, in-memory processing that accelerated iterative algorithms by orders of magnitude. Spark tried to unify batch and streaming through micro-batching, but this still added latency. Apache Flink countered this by treating batch as bounded streams, achieving true low-latency processing.

### **4.5 Modern Convergence: Cloud-Native and Lakehouse Architectures**

Today’s architectures reject the tight coupling of storage and compute that defined Hadoop. Modern systems use cheap, scalable cloud object storage with elastic compute engines. This evolution led to the Data Lakehouse [13], powered by formats like Apache Iceberg. The Lakehouse merges data lakes and warehouses, bringing ACID transactions and schema enforcement to decoupled cloud storage.

## **5 Question 4: The Rise of NoSQL**

NoSQL databases like MongoDB and Apache Cassandra have gained popularity because they address key limitations of traditional relational databases, such as MySQL and Oracle. Relational databases excel in structured data and complex transactions but struggle with scalability, flexibility, and real-time processing, which are critical requirements for modern applications like social media, IoT, and genomics [14, 15].

### **CAP Theorem and Trade-offs**

The CAP theorem (Consistency, Availability, Partition Tolerance) explains the fundamental trade-offs in distributed systems. NoSQL databases prioritize different aspects. Cassandra focuses on Availability and Partition Tolerance (AP), ensuring continuous operation even during network failures, though it may occasionally show outdated data. MongoDB offers flexibility, allowing configurations for either Consistency and Partition Tolerance (CP) or AP, depending on application needs [16, 17].

Most NoSQL systems favor AP because modern applications like social networks and e-commerce require uptime and speed over perfect consistency.

## Workloads Ill-Served by Relational Systems

Relational databases struggle with high-velocity data (e.g., real-time analytics, IoT sensor streams), unstructured or semi-structured data (e.g., genomic sequences, social media posts), and distributed environments where data is generated across multiple locations.

NoSQL databases like HBase, used in the Genome Analysis Toolkit, efficiently handle these challenges, offering faster writes, better scalability, and lower costs [18, 19].

## Overcorrection and the Return of Relational/NewSQL

While NoSQL excels in scalability, some applications, such as financial transactions, require ACID compliance, which NoSQL often sacrifices for speed. Hybrid systems like NewSQL (e.g., Google Spanner) now combine NoSQL's scalability with relational reliability, ensuring both performance and accuracy [20].

## 6 Question 5: Scalability – Theory and Practice

Scalability is a core principle in distributed systems and refers to the ability of a system to handle increasing workloads by proportionally increasing computational resources. From a theoretical perspective, scalability is fundamentally constrained by limits of parallelism. Amdahl's Law states that the maximum achievable speedup of a parallel system is limited by the fraction of computation that remains sequential. Even with a large number of processors, the sequential component becomes a bottleneck. In contrast, Gustafson's Law argues that when system size increases, workloads typically grow proportionally, allowing effective utilization of additional processors. This distinction corresponds to the difference between strong scaling, where runtime decreases for a fixed workload, and weak scaling, where workload increases proportionally with resources while runtime remains stable. Empirical studies show that architectural overhead, such as centralized coordination mechanisms, can significantly limit scalability at large scale [21].

From a technological perspective, scalability in cloud systems is mainly achieved through horizontal scaling, where additional virtual machines or containers are deployed dynamically. Early research on auto-scaling classified approaches into threshold-based rules, control theory, reinforcement learning, queuing theory, and time-series analysis [22]. Threshold-based rules are simple and widely adopted in commercial cloud platforms, but they are purely reactive and may lead to oscillation, over-provisioning, or SLA violations if thresholds are not carefully tuned [22]. More advanced approaches such as control-theoretic and queuing models attempt to optimize resource allocation by modeling system dynamics, although they require accurate system knowledge and may lack flexibility in highly dynamic environments.

Autoscaling mechanisms can generally be categorized as reactive or proactive. Reactive scaling adjusts resources based on current system metrics such as CPU utilization or request rate. Kubernetes' Horizontal Pod Autoscaler (HPA) is a typical example, scaling pods based on observed resource consumption. While reactive mechanisms are robust and relatively easy to deploy, they suffer from delayed response due to monitoring intervals and container startup times [23]. In contrast, proactive autoscaling incorporates workload prediction techniques, such as time-series forecasting or machine learning models, to allocate resources before overload occurs. Predictive autoscaling has been shown to reduce response time and improve resource efficiency compared to purely re-

active mechanisms [24]. Hybrid approaches combine reactive scale-up with predictive scale-down strategies to reduce SLA violations and avoid oscillatory behavior in cloud infrastructures [25].

With the rise of container-based virtualization and microservices, scalability has become more fine-grained and dynamic. Containers are lightweight compared to virtual machines and allow faster scaling operations, but they also introduce new orchestration complexities. Recent surveys emphasize that most current container-based autoscaling mechanisms remain reactive and rely heavily on CPU-based thresholds [23]. However, modern workloads, particularly microservices and machine learning applications, often require multi-resource scaling strategies involving CPU, memory, network bandwidth, and GPU resources. Designing autoscalers for containerized cloud and edge environments therefore requires more sophisticated, multi-metric decision mechanisms [23].

Edge and fog computing introduce additional scalability challenges. Unlike centralized cloud data centers, edge environments are heterogeneous and resource-constrained. Workloads are highly dynamic and latency-sensitive, especially in IoT applications. Traditional cloud autoscaling techniques cannot be directly applied without adaptation [23]. Proactive and hybrid autoscaling strategies become increasingly important in such contexts to prevent SLA violations while minimizing resource waste. However, accurate prediction at the edge is difficult due to limited historical data and variable network conditions.

In conclusion, scalability is theoretically bounded by limits of parallelism but practically enabled through distributed architectures and intelligent resource management. Reactive autoscaling offers simplicity and operational stability, whereas proactive and hybrid strategies improve responsiveness and cost-efficiency. The evolution from VM-based cloud systems to containerized cloud and edge environments demonstrates that scalability is no longer merely a hardware provisioning problem but a dynamic control and prediction problem requiring adaptive, data-driven approaches [22, 23].

## References

- [1] S. Shahrivari, “Beyond batch processing: Towards real-time and streaming big data,” *Computers*, vol. 3, no. 4, pp. 117–129, 2014. [Online]. Available: <https://www.mdpi.com/2073-431X/3/4/117>
- [2] S. Kulkarni, N. Bhagat, M. Fu, V. Kedigehalli, C. Kellogg, S. Mittal, J. M. Patel, K. Ramasamy, and S. Taneja, “Twitter heron: Stream processing at scale,” in *Proceedings of the 2015 ACM SIGMOD International Conference on Management of Data*, ser. SIGMOD ’15. New York, NY, USA: Association for Computing Machinery, 2015, p. 239–250. [Online]. Available: <https://doi.org/10.1145/2723372.2742788>
- [3] J. Dean and S. Ghemawat, “Mapreduce: Simplified data processing on large clusters,” in *Proceedings of the 6th Conference on Symposium on Operating Systems Design & Implementation*. USENIX Association, 2004, pp. 10–10.
- [4] A. Toshniwal, S. Taneja, A. Shukla, K. Ramasamy, J. M. Patel, S. Kulkarni, J. Jackson, K. Gade, M. Fu, J. Donham, N. Bhagat, S. Mittal, and D. Ryaboy, “Storm@twitter,” in *Proceedings of the 2014 ACM SIGMOD International Conference on Management of Data*, ser. SIGMOD ’14. New York, NY, USA: Association for Computing Machinery, 2014, p. 147–156. [Online]. Available: <https://doi.org/10.1145/2588555.2595641>
- [5] P. Carbone, A. Katsifodimos, S. Ewen, V. Markl, S. Haridi, and K. Tzoumas, “Apache flink™: Stream and batch processing in a single engine,” *Bulletin of the IEEE Computer Society Technical Committee on Data Engineering*, vol. 36, no. 4, pp. 28–38, 2015.
- [6] S. Li, P. Gerver, J. MacMillan, D. Debrunner, W. Marshall, and K.-L. Wu, “Challenges and experiences in building an efficient apache beam runner for ibm streams,” *Proceedings of the VLDB Endowment*, vol. 11, no. 12, pp. 1742–1754, 2018.
- [7] T. Akidau, R. Bradshaw, C. Chambers, S. Chernyak, R. J. Fernández-Moctezuma, R. Lax, S. McVeety, D. Mills, F. Perry, E. Schmidt, and S. Whittle, “The dataflow model: a practical approach to balancing correctness, latency, and cost in massive-scale, unbounded, out-of-order data processing,” *Proc. VLDB Endow.*, vol. 8, no. 12, p. 1792–1803, Aug. 2015. [Online]. Available: <https://doi.org/10.14778/2824032.2824076>
- [8] L. Zeng *et al.*, “An empirical evaluation of columnar storage formats: Apache parquet in the context of big data,” 2023.
- [9] M. L. Kersten *et al.*, “The researcher’s guide to the data deluge: Querying a scientific database in just a few seconds,” 2011.
- [10] M. Zaharia, M. Chowdhury, T. Das, A. Dave, J. Ma, M. McCauley, M. J. Franklin, S. Shenker, and I. Stoica, “Resilient distributed datasets: A fault-tolerant abstraction for in-memory cluster computing,” in *Proceedings of the 9th USENIX Conference on Networked Systems Design and Implementation*. USENIX Association, 2012, pp. 2–2.



- [11] F. Chang, J. Dean, S. Ghemawat, W. C. Hsieh, D. A. Wallach, M. Burrows, T. Chandra, A. Fikes, and R. E. Gruber, “Bigtable: A distributed storage system for structured data,” in *Proceedings of the 7th Symposium on Operating Systems Design and Implementation*. USENIX Association, 2006, pp. 205–218.
- [12] J. Kreps, N. Narkhede, and J. Rao, “Kafka: A distributed messaging system for log processing,” in *Proceedings of the NetDB Workshop*, vol. 11, 2011, pp. 1–7.
- [13] M. Armbrust, A. Ghodsi, R. Xin, and M. Zaharia, “Lakehouse: A new generation of open platforms that unify data warehousing and advanced analytics,” in *Proceedings of the 11th Annual Conference on Innovative Data Systems Research (CIDR)*, 2021.
- [14] X. Wu, X. Zhu, G.-Q. Wu, and W. Ding, “Data mining with big data,” *IEEE Transactions on Knowledge and Data Engineering*, vol. 26, no. 1, pp. 97–107, 2014.
- [15] C. P. Chen and C.-Y. Zhang, “Data-intensive applications, challenges, techniques and technologies: A survey on big data,” *Information Sciences*, vol. 275, pp. 314–347, 2014.
- [16] ScyllaDB. (2025) What is cap theorem? definition & faqs. [Online]. Available: <https://www.scylladb.com/glossary/cap-theorem/>
- [17] (2026) Cap theorem. Wikipedia. [Online]. Available: [https://en.wikipedia.org/wiki/CAP\\_theorem](https://en.wikipedia.org/wiki/CAP_theorem)
- [18] A. McKenna *et al.*, “The genome analysis toolkit: A mapreduce framework for analyzing next-generation dna sequencing data,” *Genome Research*, vol. 20, no. 9, pp. 1297–1303, 2010.
- [19] R. C. Taylor, “An overview of the hadoop/mapreduce/hbase framework and its current applications in bioinformatics,” *BMC Bioinformatics*, vol. 11, no. S12, 2010.
- [20] Memgraph. (2023) When to use a nosql database. [Online]. Available: <https://memgraph.com/blog/when-to-use-a-nosql-database>
- [21] M. Lunacek, J. Braden, and T. Hauser, “The scaling of many-task computing approaches in python on cluster supercomputers,” in *Proceedings of the IEEE International Conference on Cluster Computing*, 2013.
- [22] T. Llorido-Botrán, J. Miguel-Alonso, and J. A. Lozano, “A review of auto-scaling techniques for elastic applications in cloud environments,” *Journal of Grid Computing*, vol. 12, no. 4, pp. 559–592, 2014.
- [23] J. Dogani, R. Namvar, and F. Khunjush, “Auto-scaling techniques in container-based cloud and edge/fog computing: Taxonomy and survey,” *Computer Communications*, vol. 209, pp. 120–150, 2023.
- [24] L. Ju, P. Singh, and S. Toor, “Proactive autoscaling for edge computing systems with kubernetes,” in *Proceedings of the IEEE/ACM International Conference on Utility and Cloud Computing Companion*, 2021.
- [25] A. Ali-Eldin, J. Tordsson, and E. Elmroth, “An adaptive hybrid elasticity controller for cloud infrastructures,” in *Proceedings of the IEEE Network Operations and Management Symposium*, 2012, pp. 204–212.